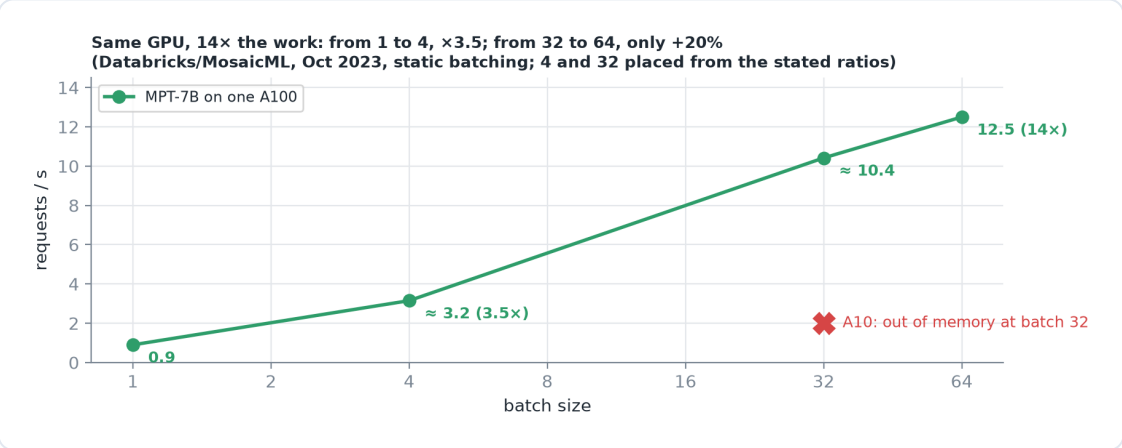
25 · Sweep Concurrency & Request Rate

Source: fanout.sh / inference-eng / 05-02 · ~9 min

IN SHORT

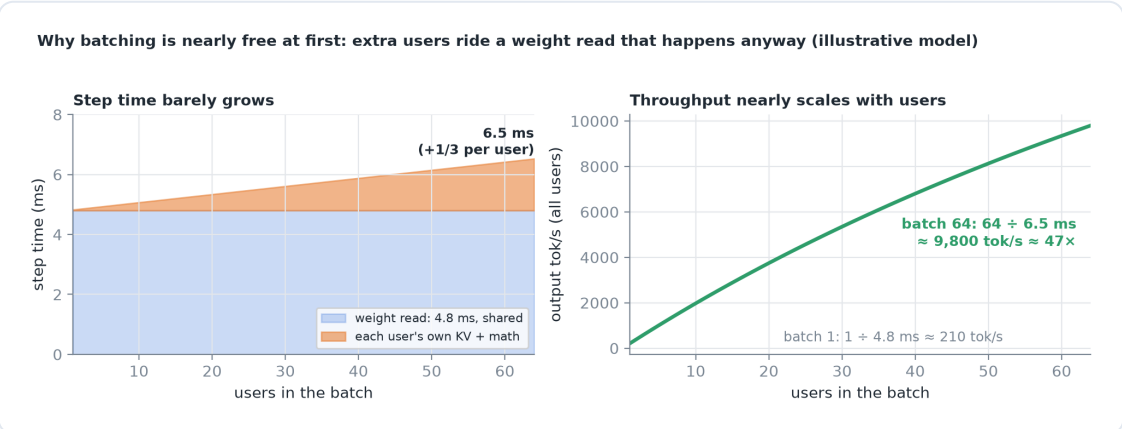
Batching is nearly free at first: extra users ride a weight read that happens anyway (illustrative: 1 user at 4.8 ms/step → 64 users at 6.5 ms gives **~47× throughput for +1/3 TPOT**; Databricks measured **14×** on one A100). It ends when per-user costs dominate or the **KV cache fills**. A **concurrency sweep** (1, 2, 4 … 256 in flight) draws the latency-throughput curve: **flat → knee → wall**, and your SLO line picks the operating point (the rightmost dot under it). A **request-rate sweep** with random arrivals shows TTFT exploding near capacity and, past it, **growing for as long as you run**. Peak throughput is not capacity: capacity is the most load that still meets the targets.

1. The free lunch



In 2023 MosaicML at Databricks served MPT-7B on one A100: 0.9 req/s one at a time, 12.5 req/s at batch 64. On a smaller A10 the climb stopped at batch 32: out of memory.

Start from the single-request baseline: one user on the H100, each decode step ≈ **4.8 ms**, almost all of it reading weights. Add a second user: the step still reads the weights **once**, and that one read now makes two tokens. The extra user only adds their own keys and values to read, plus a little arithmetic.

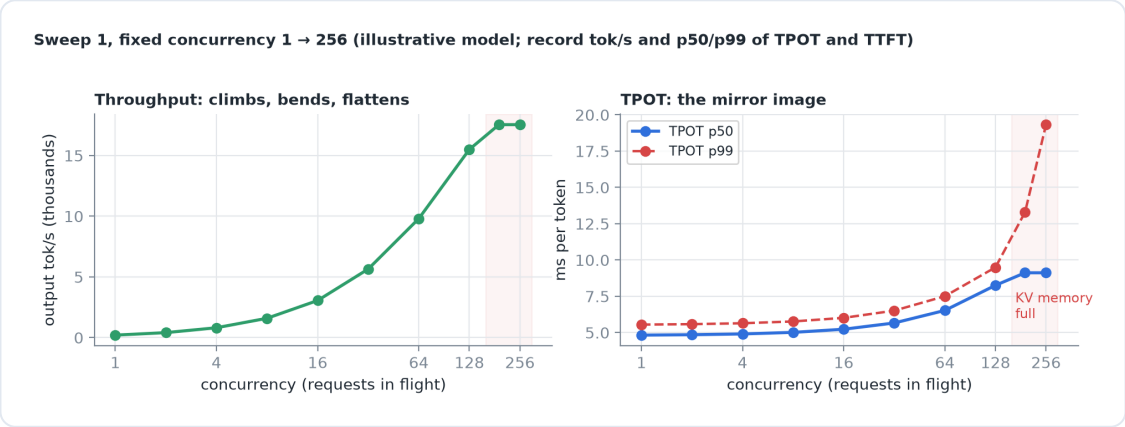


Illustrative model: 4.8 ms plus a small slice per user. At 64 users a step takes ~6.5 ms: each user's TPOT rises by a third, but the GPU makes 64 tokens per step instead of one.

It ends in two ways: the **per-user slices add up** until they dominate the step, or the **KV cache fills** GPU memory and new users have to wait.

2. Sweep 1: fixed concurrency

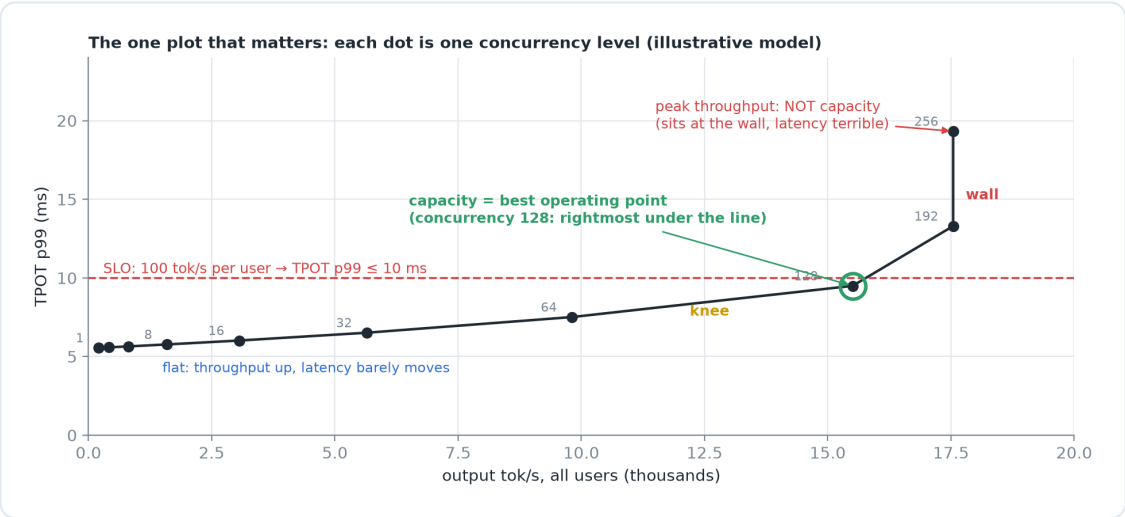
Keep exactly N requests in flight; when one finishes, the next starts. Run N = 1, 2, 4, 8 … doubling up to 256 or until something breaks. At each level record **total output throughput** and the **median and P99 of TPOT and TTFT**.



Throughput climbs steeply, bends, flattens; TPOT is the mirror image. Databricks saw the same shape: 1 → 4 users gave 3.5× the throughput, 32 → 64 only about 20% more.

3. The one plot that matters

Put throughput on the x-axis and TPOT on the y-axis, one dot per concurrency level, and read left to right.



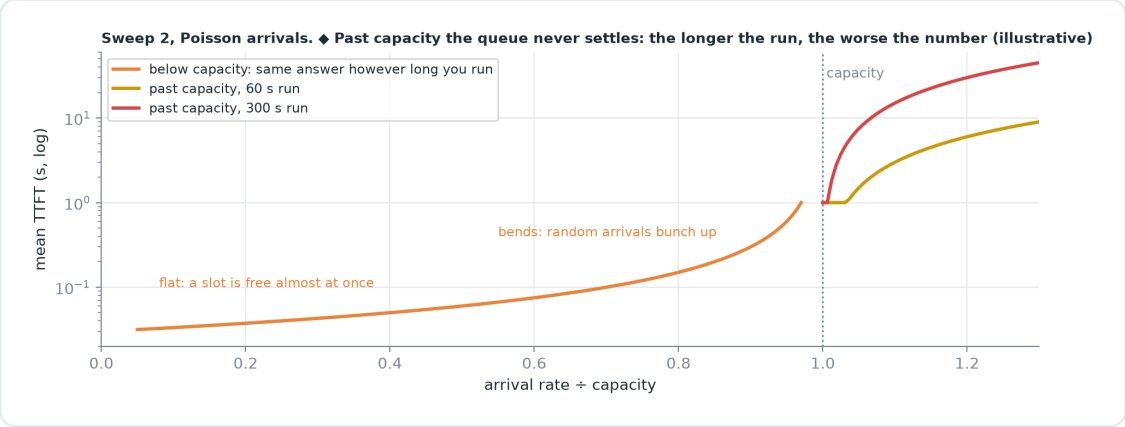
Region	What happens
Flat	Throughput grows a lot, latency barely moves. You should almost always be at least this far along
Knee	Each extra user still adds throughput, but costs every other user a noticeable slowdown
Wall	Throughput stops growing; latency keeps climbing as requests wait for memory or a slot

KEY IDEA

Your latency target picks the spot. If every user must see ≥ **100 tokens/s**, that's **TPOT ≤ 10 ms at P99**: draw the line, and the best operating point is the **rightmost dot still under it**.

4. Sweep 2: fixed request rate

The concurrency sweep only sends a new request when an old one finishes, so it slows down when the server does. Real users arrive whenever they like. So the second sweep fixes the **arrival rate** (random arrivals averaging, say, 10/s) and raises it, watching **TTFT**.



At low rates TTFT is flat: a slot is free almost at once. Near capacity it bends sharply as random arrivals bunch up. Past capacity the queue grows without limit.

INTUITION

If your result **depends on how long you ran the benchmark**, you're past capacity.

5. Running both with vLLM

```
vllm bench serve \
  --max-concurrency 32 \
  --request-rate 10 --burstiness 1 \
  --dataset-name random --random-input-len 512 \
  --random-output-len 256 --ignore-eos \
  --percentile-metrics ttft,tpot,e2el \
  --metric-percentiles 50,90,99 \
  --goodput ttft:500 tpot:10
```

- `--max-concurrency N` for sweep 1; `--request-rate` with `--burstiness 1` (Poisson-style) for sweep 2.
- Fixed shape: random 512-token prompts, 256 output tokens, ignore EOS. `--goodput` counts requests that met your targets.
- `vllm bench sweep` runs a whole grid; **GuideLLM** has a sweep mode that finds the range automatically.

CAUTION

If you don't set a request rate it's **infinite**: every request is sent at time zero. That's one burst, not a steady rate.

6. How sweeps quietly go wrong

Pitfall	Effect
Too few requests	At high rates a short run never reaches steady state: you measure the ramp-up
A weak client	The sender can't keep up: you benchmark the client. Compare achieved vs requested rate
Unrealistic prompts	Random prompts never share a prefix, so prefix caching never helps; real chat often does
Averages	The median looks fine while P99 has blown through the target

CAUTION

The highest throughput on the curve **is not your capacity**: it sits at the wall, where latency is terrible. Capacity is the highest load that still meets the latency targets (goodput), always left of the peak.

7. Recap



Interview cheat sheet

Revise from these pages alone. ◆ = my addition beyond the video; everything else is from the lesson. Closed vs open loops, Little's law and the queueing knee are in 21; goodput in 23; the batch-size floor model in 16 (derivation 3).

Say it in 30 seconds

> *At batch 1 a decode step is almost all weight reading, so extra users nearly ride for free: in an illustrative model 64 users take 6.5 ms instead of 4.8, 47× the throughput for a third more TPOT, and Databricks saw 14× on an A100. It stops when per-user KV reads dominate or the cache fills. I'd sweep concurrency 1 to 256 and plot P99 TPOT against total tokens/s: flat, knee, wall. The SLO picks the point, e.g. 100 tokens/s per user is TPOT ≤ 10 ms, so take the rightmost dot under that line. Then sweep Poisson request rates and watch TTFT; if it depends on run length, you're past capacity. Peak throughput isn't capacity.*

Only numbers worth memorizing

- **Databricks MPT-7B, A100:** 0.9 → 12.5 req/s from batch 1 to 64 (**14×**); 1→4: ×3.5, 32→64: +20%.

Derive, don't memorize

1. The free lunch: step = shared read + per-user slice

$$\text{tok/s} = \frac{B}{t_{\text{step}}}: \quad \frac{1}{4.8\ \text{ms}} \approx 210 \quad \frac{64}{6.5\ \text{ms}} \approx 9800 \quad (47\times) \qquad \text{TPOT}: \frac{6.5}{4.8} \approx 1.35$$

> **INTUITION**
>
> ◆ The slice is one user's KV read: (6.5 − 4.8) ÷ 64 ≈ 0.027 ms ≈ 128 KB × ~640 tokens ÷ 3.35 TB/s. Longer contexts grow the slice, so the knee comes sooner.

2. SLO → TPOT line → operating point

$$100\ \text{tok/s per user} \Rightarrow \text{TPOT}_{\text{p99}} \le \frac{1}{100} = 10\ \text{ms}$$

3. Diminishing returns as marginal efficiency ◆

$$1 \to 4: \frac{3.5}{4} \approx 88\%\ \text{of ideal} \qquad 32 \to 64: \frac{1.2}{2} = 60\%\ \text{of ideal}$$

4. Past capacity, TTFT grows with run length ◆

The queue grows at arrivals minus completions; the newest request waits for everyone ahead.

$$\lambda = 12,\ \mu = 10\ \text{req/s}: \quad \text{queue} = 2 \times 60\ \text{s} = 120 \Rightarrow \text{wait} \approx \frac{120}{10} = 12\ \text{s} \qquad 300\ \text{s}: 60\ \text{s}$$

Concurrency sweep vs rate sweep

	Concurrency sweep	Rate sweep
Fixes	Requests in flight (1 … 256)	Arrival rate (Poisson)
Watch	Total tok/s vs P99 TPOT	TTFT (P50, P99)
Answers	Max throughput, operating point under SLO	What real arrivals do near capacity
vLLM flag	`--max-concurrency N`	`--request-rate R --burstiness 1`
Blind spot	Slows down with the server	Needs long runs to reach steady state

Symptom → diagnosis

Symptom	Diagnosis
TTFT gets worse the longer the test runs	Past capacity: queue growing without limit
Achieved rate below requested	Server or client can't keep up
Huge TTFT in a "low load" rate test	No rate set (infinite): every request sent at t = 0
Prod beats the benchmark on TTFT	Real prompts share prefixes; random ones don't
Out of memory partway up the sweep	KV cache full (the A10 at batch 32)

Rapid-fire Q&A

Question	Crisp answer
Why is batching nearly free at first?	Users share one weight read; each adds only its KV and math (derivation 1).
Where does it stop?	Per-user slices dominate, or KV memory fills.
How to pick the operating point?	Rightmost dot under the P99 TPOT line set by the SLO.
Is peak throughput your capacity?	No: capacity is max load meeting targets, left of the peak.
Why two sweeps?	Concurrency finds the curve; rate shows real random arrivals.

TRAP

- Running the rate sweep without setting `--request-rate` (infinite burst).
- Reporting capacity at the throughput peak.
- Judging a sweep by medians only.